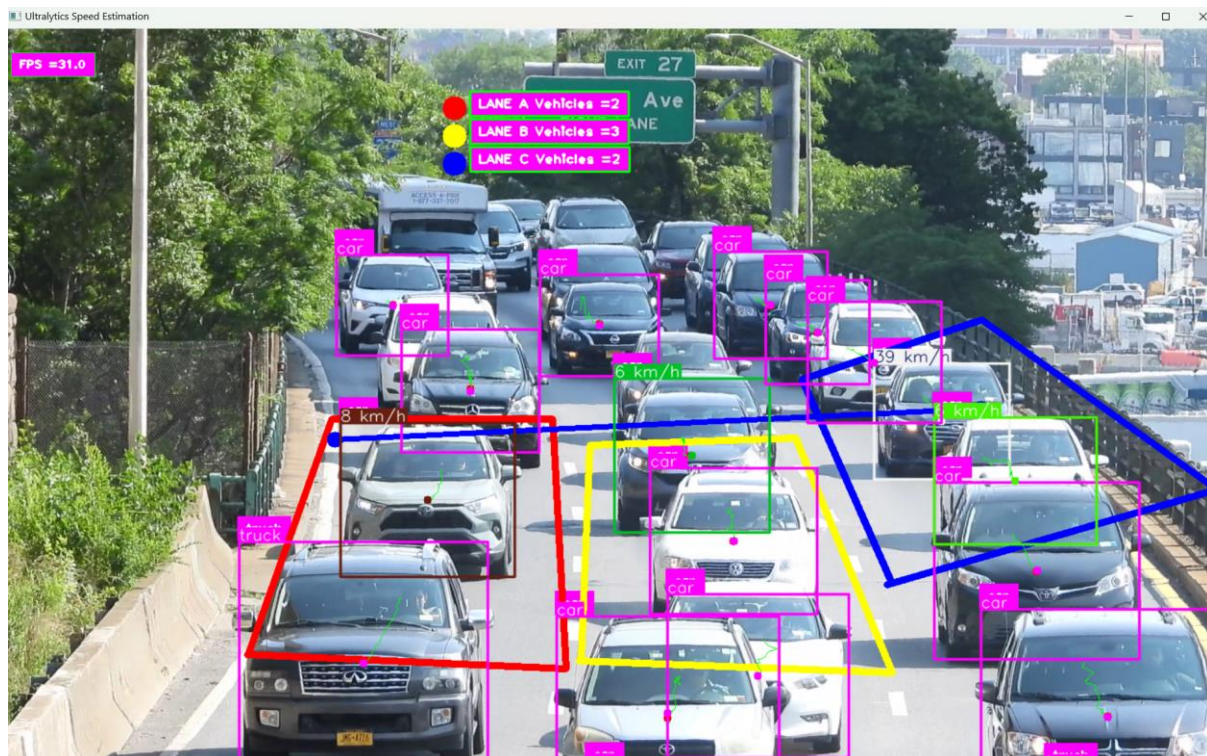


Github Link: https://github.com/Cheonny/Sunway_Assessment

Challenge 1: Traffic Analysis

The detection model used is YOLOv5n. In the first video streaming, there is a busy road on three lanes. The code snippet allows detecting, counting of vehicles each lane and estimating their moving speed. Based on the average of estimated speed, analysis of lane occupancy and traffic congestion can be conducted. For future work, it should be able to detect emergency and non-emergency vehicles and track the abnormal scenarios. For instance, a vehicle moves with abnormal speed (extremely slow or fast), broken, or accidents so that the parties can respond to it immediately.

Busy Road Screenshot:

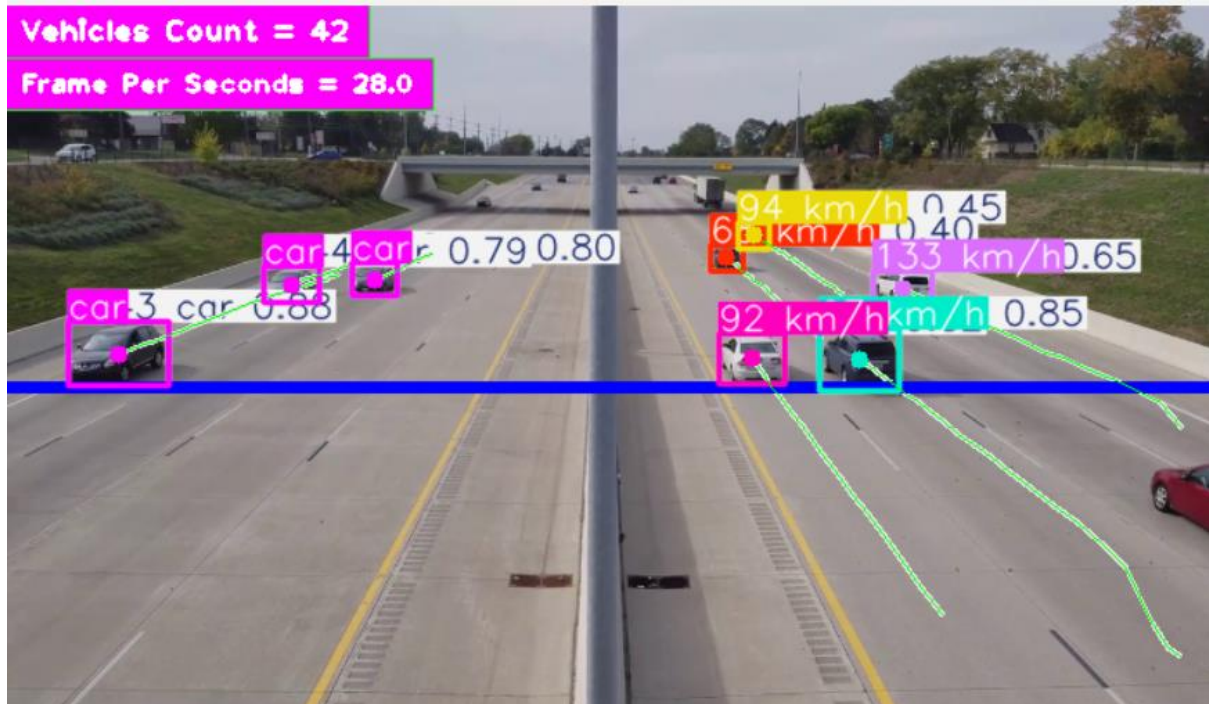


The detection model successfully detects the type of vehicles and count the vehicles passed through the drawn region. Other than that, the speed estimator allows the program to estimate the speed of vehicles accurately.

By using YOLOv5s model, the fps rate is low (16fps) which lead to some delays. YOLOv5n and YOLOv8n (31fps) performs well in terms of inference speed.

Highway Video

Another scenario of detecting and tracking vehicles is on the highway. In this video streaming, the inference speed is more near to the real time inference especially using YOLOv5n and YOLOv8n model. Compared to YOLOv5s, they demonstrate better performance in inference speed and FPS rate.



The speed of the vehicles are estimated accurately, where they moves approximately 60km/h and above on the highway road.

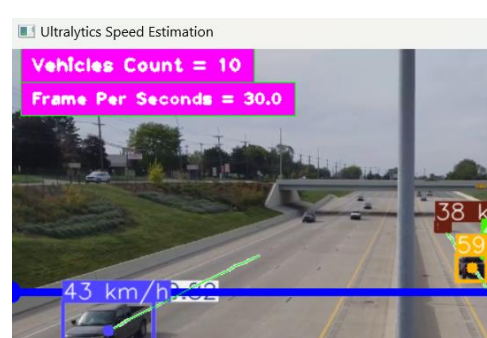
	Highway Road Video			Busy Road Video		
Performance	Yolov5s	Yolov5n	Yolov8n	Yolov5s	Yolov5n	Yolov8n
Frame Per Seconds	16-20	24-30	22-33	16-19	28-33	27-33

The higher the FPS rate, the lower the inference speed (faster). YOLOv8n performs the best among these YOLO models. FPS formula:

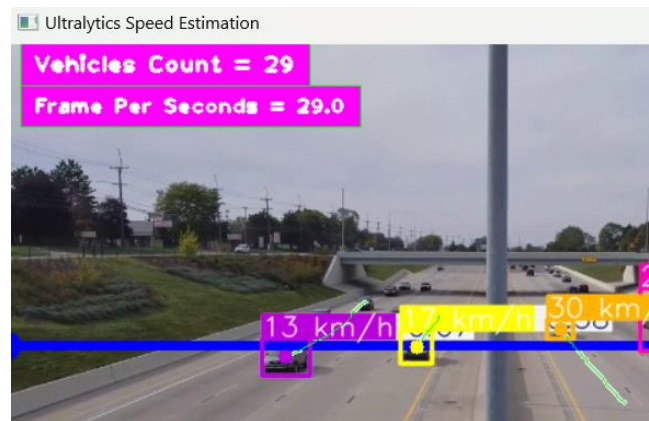
$$fps = \frac{1000ms}{inference\ speed\ (ms)}$$



Yolov5s



Yolov5n



Yolov8n

Some screenshots of code snippet:

```

50 # Loop through the video frames
51 while cap.isOpened():
52     # Read a frame from the video
53     success, frame = cap.read()
54     frame = cv2.resize(frame, (720,420)) # Resize input frames
55     if success:
56
57         # Run YOLOv8 tracking on the frame, persisting tracks between frames
58         results = model.track(frame, persist=True)
59
60         # Get the boxes and track IDs
61         boxes = results[0].boxes.xywh.cpu()
62         track_ids = results[0].boxes.id.int().cpu().tolist()
63
64         # Visualize the results on the frame
65         annotated_frame = results[0].plot()
66
67         # Plot the tracks
68         for box, track_id in zip(boxes, track_ids):
69
70             x, y, w, h = box
71             cx, cy = int(x + w / 2), int(y + h / 2)
72             cv2.circle(frame, (cx,cy), 5, (0,0,255),-1)
73             # Find out if that object was detected already
74
75             if track_id not in track_history:
76                 count += 1
77
78             track = track_history[track_id]
79             track.append((float(x), float(y))) # x, y center point
80             if len(track) > 30: # retain 90 tracks for 90 frames
81                 track.pop(0)
82
83             # Draw the tracking lines
84             points = np.hstack(track).astype(np.int32).reshape((-1, 1, 2))
85             cv2.polylines(annotated_frame, [points], isClosed=False, color=(230, 230, 230), thickness=2)
86
87             inference = results[0].speed['inference']
88             fps = np.ceil(1000/int(inference))
89             cvzone.putTextRect(annotated_frame, f'Frame Per Seconds = {fps}', [20,50], thickness=2, scale=1)
90             im0 = speed_obj.estimate_speed(annotated_frame, results, (255, 0, 0))

```

Original video link: https://drive.google.com/file/d/1Mdf1BywBgkwmVrRd8I3F8H_FPR-r5xRI/view?usp=sharing

Challenge 2: Image Processing

The images chosen in this challenge are grainy. The purpose of the code snippet below is improving the quality of image using denoising, smoothing, and sharpening techniques.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def image_enhancement(image):
    # Convert the image to grayscale (optional, if it's a colored image)
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Apply Non-Local Means Denoising
    denoised_image = cv2.fastNlMeansDenoisingColored(image, None, 10, 10, 7, 21)

    # Optional: Apply a Gaussian Blur to further smooth the image
    blurred_image = cv2.GaussianBlur(denoised_image, (3, 3), 0)

    # Sharpen the image using a kernel
    # Define a kernel for sharpening
    sharpening_kernel = np.array([[-1, -1, -1],
                                   [-1, 9, -1],
                                   [-1, -1, -1]])

    sharpened_image = cv2.filter2D(blurred_image, -1, sharpening_kernel)

    return sharpened_image
```

The code provided in the image performs image enhancement. The image is converted into grayscale and is performed non-local means denoising method to reduce noise. Other than that, gaussian blur is applied for smoothing and lastly sharpening kernel is applied on the image.

Result:

Original Grainy Image



Enhanced Image



Original Grainy Image



Enhanced Image



Two grainy images are tested. Both processed images show a significant enhancement of image quality. However, there are some lost details on the second image (e.g. over-smoothed grass background and white dresses).

Challenge 3: RAG Chatbot

Integrating Retrieval-Augmented Generation (RAG) allows for the customization of language models to meet specific needs by combining document retrieval with text generation. This chatbot serves as personalized chatbot based on the information fed (website links or file upload). One key use case is that the chatbot serves as an assistant for organizational knowledge, enhancing both the efficiency and accuracy of retrieving information within the organization.

The chatbot user interface is programmed using Streamlit and backend is programmed integrating with LangChain, Ollama, and Chroma. The program retrieves the relevant documents from the source and splits the text into chunks for retrieval. Vector embeddings are used to represent the documents. Chroma is used as the vector database to store and retrieve document embeddings while the local embedding model, 'nomic-embed-text' is used to provide embeddings for the documents, enhancing the retrieval process.

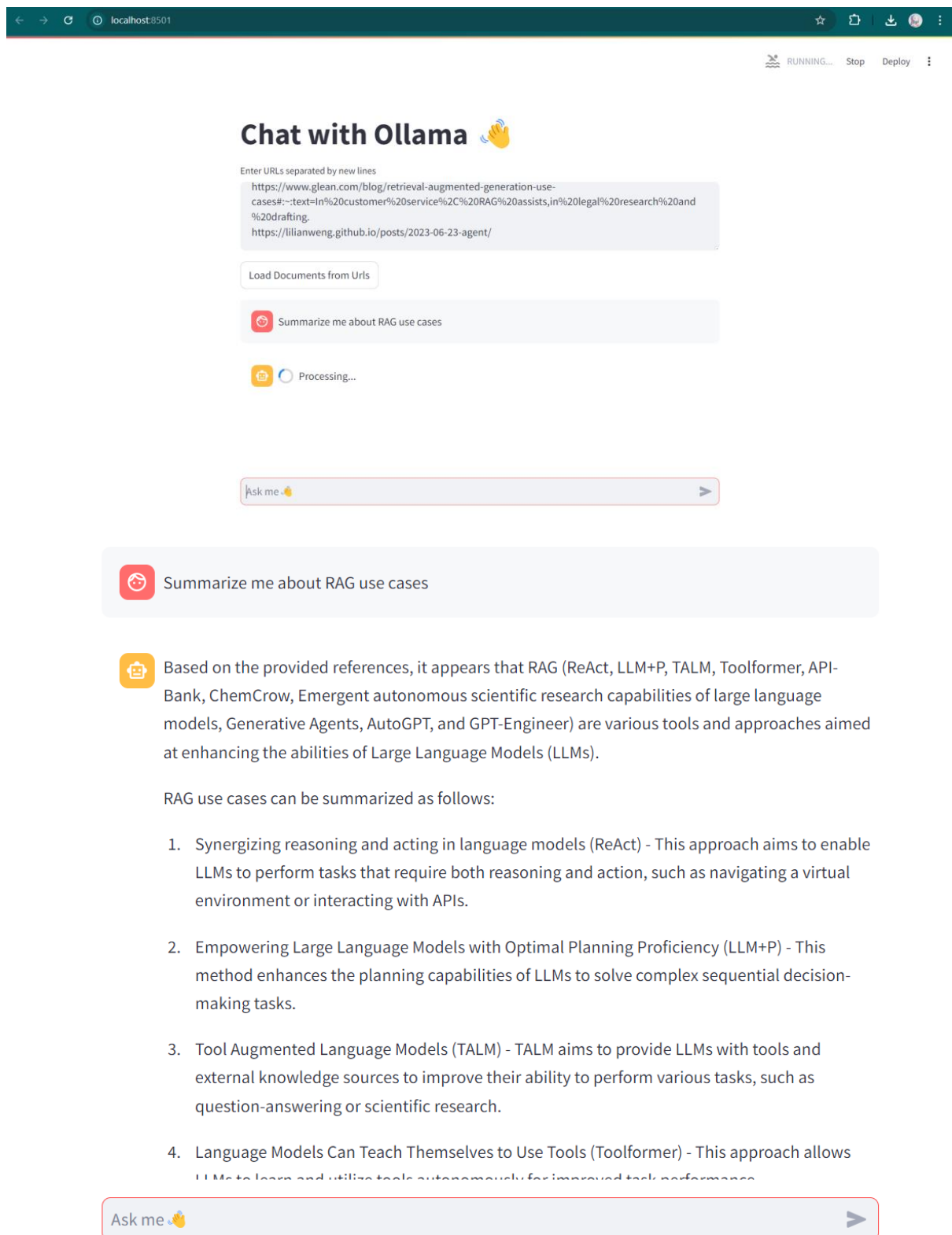
Document Retrieval and Vector Representation

```
11 def vector_db(urls):
12     # Convert string of URLs to list
13     urls_list = urls.split("\n")
14     docs = [WebBaseLoader(url).load() for url in urls_list]
15     docs_list = [item for sublist in docs for item in sublist]
16
17     # PDF processing
18     # loader = PyPDFLoader(pdf_path)
19     # docs_list = loader.load() # Load PDF as list of documents
20
21     #split the text into chunks
22     text_splitter = CharacterTextSplitter.from_tiktoken_encoder(chunk_size=7500, chunk_overlap=100)
23     doc_splits = text_splitter.split_documents(docs_list)
24
25     #convert text chunks into embeddings and store in vector database
26     vectorstore = Chroma.from_documents(
27         documents=doc_splits,
28         collection_name="rag-chroma",
29         embedding=embeddings.OllamaEmbeddings(model='nomic-embed-text')
30     )
31     return vectorstore
```

RAG is then performed using model “Mistral” and retrieved vectors based on the user prompt.

```
33 def process_input(question,vectorstore):
34     model_local = Ollama(model="mistral")
35     retriever = vectorstore.as_retriever()
36
37     #perform the RAG
38
39     after_rag_template = """Answer the question based only on the following context:
40 {context}
41 Question: {question}
42 """
43     after_rag_prompt = ChatPromptTemplate.from_template(after_rag_template)
44     after_rag_chain = (
45         {"context": retriever, "question": RunnablePassthrough()}
46         | after_rag_prompt
47         | model_local
48         | StrOutputParser()
49     )
50     return after_rag_chain.invoke(question)
```

Screenshots of User Interface:



The response shows that the summarized the data from these two links but mostly retrieved from one link which is more informative.

Bonus challenge:

Idea: Install adaptive traffic lights within Sunway City region to adjust their timings based on emergency vehicle priority. Integration between traffic signals, emergency services, and traffic management systems to allow ambulance transports faster. Ambulances equipped with GPS and real-time adjustments of traffic signals.